

UNIVERSIDAD PANAMERICANA
Facultad De Ingeniería y Ciencias Aplicadas
Ingeniería en Sistemas y Tecnologías de la Información y la Comunicación
Programación V/ Base de datos 2



Proyecto HIPSTAGRAM – Fase 3
(DISEÑO DE ARQUITECTURA)

Integrantes:

Douglas Esaú Catú Otzoy (000140060)
Kevin Eliazar Herrera Alburez (000113637)
Priscila Andrea Güitz del Cid (000139626)
Carlos Enrique Otoniel Bautista Sarazua (000133047)
Herbert David Estrada Trujillo (000141126)

Chimaltenango, Chimaltenango, junio 2026

DOCUMENTO DE DISEÑO DE ARQUITECTURA

Proyecto HIPSTAGRAM

Plataforma de Red Social con Microservicios

Versión	3.0
Fecha	junio 2026
Equipo	Equipo de Desarrollo — 5 integrantes
Curso	Proyecto General — Ing. Mario Obed Morales Guitz

índice

1. Introducción	1
2. Diagrama de Componentes	2
3. Descripción de Componentes	4
4. Vista Lógica — Arquitectura de Componentes de Software	5
5. Diagrama de Capas.....	8
6. Descripción de Seguridad y Credenciales.....	10
6.1 Hashing de Contraseñas con bcrypt	10
6.2 Ciclo de Vida de los Tokens JWT	10
6.3 Control de Acceso por Roles (RBAC).....	11
6.4 Validación y Sanitización de Inputs.....	12
6.5 Gestión de Secretos y Variables de Entorno	12
6.6 Checklist OWASP Top 10 — Estado de Implementación.....	13
7. Diagrama de Seguridad.....	14
7.1 Flujo de Autenticación y Autorización	14
7.2 Flujo de Registro e Inicio de Sesión	15
7.3 Flujo de Publicación con Moderación	16
8. Vista de Implementación	17
8.1 Descripción General.....	17
8.2 Catálogo de Microservicios — Especificación Completa	17
8.3 Diagrama de Implementación	18
8.4 Estructura del Repositorio — GitFlow	19
8.5 Dockerfile — Patrón Multi-Stage Build	19
9. Vista de Puesta en Marcha.....	21
9.1 Estrategia de Ambientes	21
9.2 Infraestructura AWS — Detalle de Servicios	21
9.3 Pipeline CI/CD — Jenkinsfile.....	22
9.4 GitFlow — Estrategia de Ramas.....	23
9.5 Pasos de Puesta en Marcha — Primera Vez	23
10. Vista de Datos.....	24
10.1 Diagrama Entidad-Relación (ERD)	24
10.2 Índices y Estrategia de Optimización.....	25
11. Diccionario de Datos	26
11.1 Tabla: users	26
11.2 Tabla: posts	27
11.3 Tabla: votes.....	27

11.4 Tabla: comments.....	27
11.5 Tablas: hashtags y post_hashtags.....	28
11.6 Tabla: audit_logs.....	28
11.7 Tablas: refresh_tokens y banned_words	29
12. Definición de Roles de la Aplicación	30
12.1 Roles de Aplicación	30
12.2 Endpoints por Rol — USER	30
12.3 Endpoints Exclusivos — ADMIN	31
12.4 Roles de Base de Datos (DBA).....	32
12.5 Códigos de Respuesta HTTP — Estándar del Sistema	32
13. Prototipo No Funcional	33
14. Alcance y Limitaciones de la Aplicación.....	34
14.1 Alcance Funcional Completo.....	34
14.2 Limitaciones Técnicas y de Alcance.....	35
14.3 Tabla de Riesgos del Proyecto	35
14.4 Restricciones del Sistema.....	36

1. Introducción

Esta especificación técnica detalla el diseño estructural y la infraestructura tecnológica del sistema Hipstagram, una plataforma distribuida orientada a la gestión, publicación y descubrimiento de contenido multimedia. Este artefacto constituye el entregable P3 correspondiente a la Fase 1 del curso de Programación V / Base de Datos 2 de la Universidad Panamericana, desarrollado bajo la supervisión del Ing. Mario Obed Morales Guitz. El propósito fundamental de este diseño es establecer una línea base formal que exponga las decisiones arquitectónicas mediante un modelo de múltiples vistas: lógica, de componentes, de datos, de seguridad, de implementación y de despliegue. Esta abstracción por capas garantiza una comprensión transversal y estandarizada para los distintos actores involucrados, incluyendo equipos de desarrollo, administración de bases de datos y evaluación académica.

A nivel lógico, la plataforma implementa un patrón de arquitectura basada en microservicios, altamente desacoplada y contenerizada mediante Docker, donde cada servicio obedece al principio de responsabilidad única. El enrutamiento perimetral se orquesta a través de un API Gateway centralizado que funge como único punto de contacto para la resolución de solicitudes externas. Este componente de frontera es el encargado de inyectar las políticas de seguridad mediante autenticación sin estado utilizando JSON Web Tokens (JWT) y autorización estructurada por Control de Acceso Basado en Roles (RBAC). Por su parte, la topología de persistencia diseñada es híbrida; el modelo transaccional y relacional es gestionado a través de PostgreSQL alojado en AWS RDS, mientras que el almacenamiento de objetos binarios, como las imágenes, se delega a Amazon S3.

En cuanto a la automatización y el despliegue, el ciclo de vida del software integra un *pipeline* de Integración y Entrega Continua (CI/CD) orquestado con Jenkins, el cual está respaldado por análisis estático de código fuente a través de SonarQube para asegurar la calidad y seguridad del *build*. El aprovisionamiento en entornos productivos se ejecuta sobre AWS ECS Fargate, garantizando un modelo de computación elástico para contenedores. Finalmente, toda la arquitectura fue fundamentada bajo los paradigmas de separación de responsabilidades, defensa en profundidad, escalabilidad horizontal y observabilidad integral mediante *logging* de auditoría. Todo el ecosistema adopta el enfoque de Infraestructura como Código (IaC), asegurando que los entornos de desarrollo sean deterministas y completamente reproducibles mediante la ejecución de un único manifiesto de Docker Compose.

2. Diagrama de Componentes

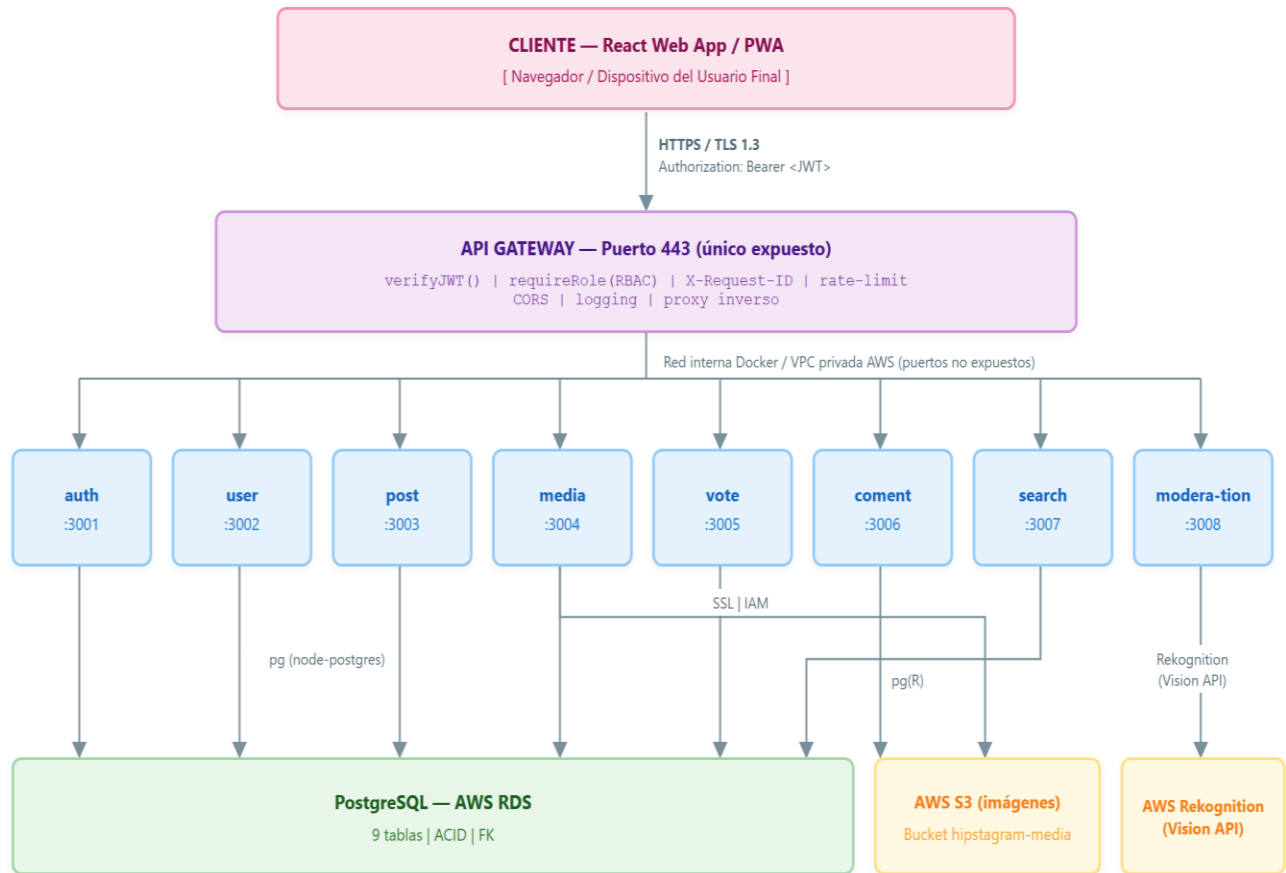
Esta especificación detalla la topología de red y la arquitectura de componentes del ecosistema Hipstagram. El diseño establece una frontera de seguridad perimetral estricta mediante la exposición exclusiva del API Gateway (operando sobre el puerto seguro 443) hacia las interfaces de consumo (React Web App / PWA). Esta configuración garantiza que todas las comunicaciones externas se encripten bajo el protocolo TLS 1.3 y requieran autorización explícita mediante tokens portadores (Bearer JWT).

El API Gateway funge como un proxy inverso y punto único de entrada, asumiendo la responsabilidad transversal de interceptar las peticiones para ejecutar la validación de identidad (verifyJWT), aplicar las políticas de Control de Acceso Basado en Roles (requireRole), mitigar riesgos mediante la limitación de tasa de peticiones (rate-limiting), gestionar las políticas CORS y asegurar la trazabilidad inyectando identificadores únicos (X-Request-ID).

Una vez superados los filtros perimetrales, el tráfico es enrutado hacia la capa de aplicación. A nivel de infraestructura interna, la lógica de negocio se encuentra altamente desacoplada en ocho microservicios autónomos (auth, user, post, media, vote, coment, search, moderation). Estos componentes operan de manera completamente confinada dentro de una red privada virtual (AWS VPC o subred aislada en Docker), manteniendo sus puertos de ejecución (3001 al 3008) inaccesibles desde el exterior de la red (Zero Trust interno).

Finalmente, la capa de aplicación se integra de forma segura con los recursos de infraestructura subyacente: consumiendo los microservicios de inteligencia artificial (AWS Rekognition) para la moderación automatizada, delegando el almacenamiento de objetos binarios a los buckets de Amazon S3, y ejecutando transacciones ACID altamente consistentes sobre el motor relacional PostgreSQL (alojado en AWS RDS) mediante controladores especializados (node-postgres).

El cliente React se comunica exclusivamente con el API Gateway. Ningún microservicio interno es accesible directamente desde el exterior. El media-service es el único con acceso directo a AWS S3 mediante el SDK de AWS autenticado con IAM. El moderation-service utiliza AWS Rekognition a través del mismo mecanismo IAM. Todos los demás microservicios acceden exclusivamente a PostgreSQL.



3. Descripción de Componentes

En estricto apego al paradigma de microservicios y al Principio de Responsabilidad Única (SRP), la arquitectura segmenta la lógica de negocio en unidades de despliegue funcionalmente autónomas y altamente desacopladas. La siguiente matriz de especificación detalla la topología del sistema, documentando el propósito operativo, el *stack* tecnológico subyacente, la asignación de puertos de red y las responsabilidades transaccionales clave para cada nodo de la infraestructura.

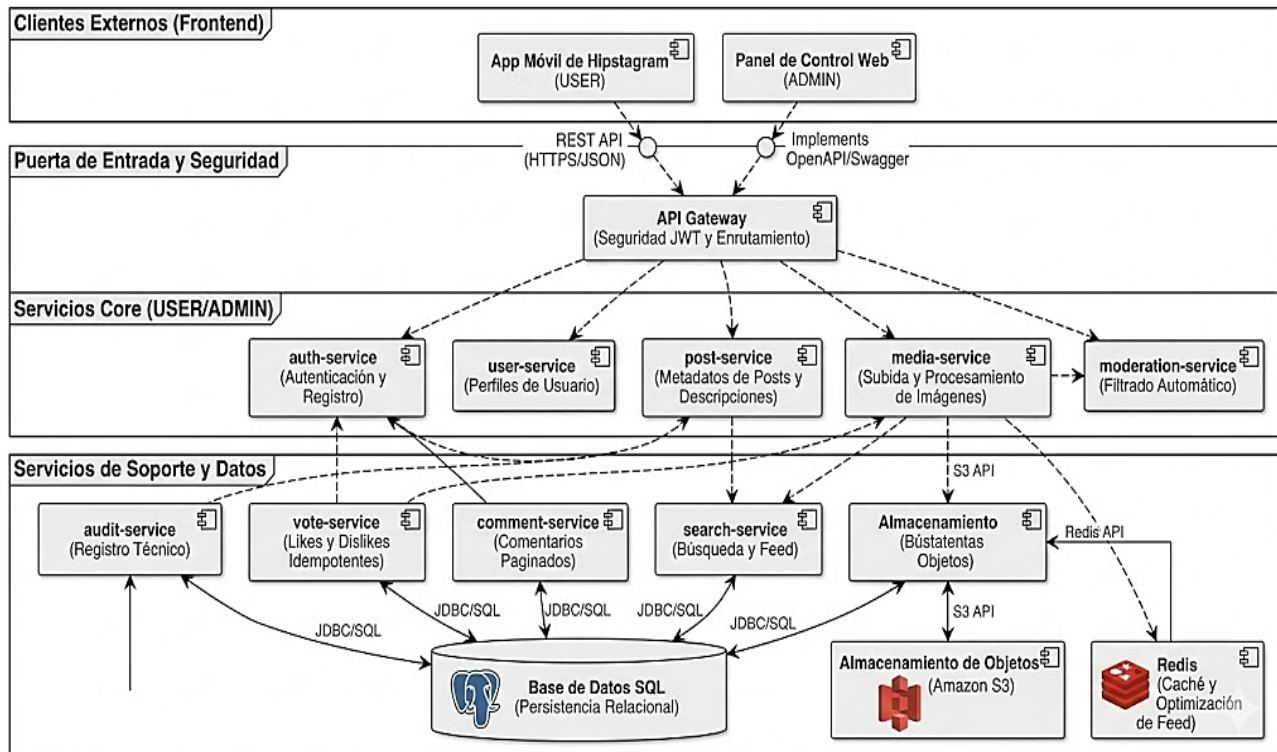
Componente	Puerto	Tecnología	Responsabilidad y características clave
React Frontend	:443	React + Vite	Interfaz de usuario principal. Maneja login, feed, publicaciones, votos, comentarios y búsqueda. Comunica exclusivamente con el API Gateway vía HTTPS. Almacena accessToken en memoria y refreshToken en cookie HttpOnly.
API Gateway	:3000	Node.js + Express	Único punto de entrada expuesto al exterior (puerto 443 en producción). Verifica firma JWT, extrae userId y role, aplica RBAC por ruta, genera X-Request-ID (UUIDv4), aplica rate-limiting (100 req/min/IP), configura CORS y reenvía al microservicio correspondiente. Stateless.
auth-service	:3001	Node.js + bcrypt + jsonwebtoken	Registro (validación Joi + bcrypt hash 12 rondas), login (bcrypt.compare + generación JWT 15min + refreshToken 7 días), logout (invalidación en BD), renovación de token (/auth/refresh). Toda acción queda en audit_logs.
user-service	:3002	Node.js + pg	Consulta y actualización de perfil de usuario. ADMIN: activar/desactivar cuentas, listar usuarios con filtros paginados, consultar audit_logs. Lee las tablas users y audit_logs.
post-service	:3003	Node.js + pg	Crear publicación con transacción COMMIT/SAVEPOINT (posts + post hashtags + audit_logs). Eliminar publicación (solo autor, maneja SQLSTATE 23503). Feed ordenado por likes_count DESC paginado. Modo Explorar. Detalle de publicación.
media-service	:3004	Node.js + AWS SDK v3	Recibe imagen multipart/form-data, la sube a AWS S3 con PutObjectCommand, llama a AWS Rekognition DetectModerationLabels con el objeto S3. Si detecta contenido inapropiado retorna HTTP 422. Si pasa, retorna la URL pública/firmada.
vote-service	:3005	Node.js + pg	Registrar like/dislike con INSERT ... ON CONFLICT (user_id, post_id) DO UPDATE. Actualiza likes_count y dislikes_count en posts de forma atómica. Previene auto-voto. Registra en audit_logs.
comment-service	:3006	Node.js + DOMPurify	Crear comentario con validación (1-500 chars) y sanitización DOMPurify. Listar comentarios paginados por publicación ordenados ASC. Eliminar (solo autor o ADMIN). FK RESTRICT detectada y retornada como HTTP 409.
search-service	:3007	Node.js + pg full-text	Búsqueda por hashtag exacto (JOIN post_hashtags+hashtags). Búsqueda full-text (to_tsvector/plainto_tsquery sobre description). Modo Explorar (ORDER BY likes_count DESC). Todos los resultados paginados con LIMIT/OFFSET. Solo lectura.
moderation-service	:3008	Node.js + AWS SDK v3	Endpoint interno (no expuesto al exterior). Recibe postId desde post-service, consulta banned_words en BD, valida hashtags. Retorna resultado de moderación. Se invoca de forma síncrona antes del COMMIT de la publicación.

4. Vista Lógica — Arquitectura de Componentes de Software

Esta abstracción estructural expone la topología interna del sistema Hipstagram, detallando la organización, responsabilidades funcionales y contratos de interfaz de sus componentes de software, con total independencia de su infraestructura de despliegue físico. El diseño adopta un patrón de arquitectura orientada a microservicios, estratificando el ecosistema en capas coherentes de responsabilidad funcional:

- **Perímetro de Consumo (Clientes Externos):** Compuesto por las interfaces de usuario finales (App Móvil para el rol USER y Panel de Control Web para el rol ADMIN). Estos artefactos consumen los recursos del sistema de manera exclusiva a través de una API RESTful (operando sobre HTTPS/JSON) fundamentada en contratos estandarizados mediante OpenAPI/Swagger.
- **Frontera de Seguridad y Enrutamiento:** Orquestada por un API Gateway centralizado que funge como el único punto de exposición lógica. Su responsabilidad principal es interceptar el tráfico entrante, aplicar las políticas de validación de identidad (seguridad mediante JWT) y resolver el enrutamiento dinámico hacia la malla de microservicios subyacente.
- **Capa de Negocio (Servicios Core y de Soporte):** La lógica del dominio se encuentra altamente desacoplada en unidades funcionales autónomas. Los servicios principales (auth, user, post, media, moderation) encapsulan las transacciones críticas de identidad y ciclo de vida del contenido. Paralelamente, los servicios de soporte (audit, vote, comment, search) gestionan las mecánicas de interacción social, la trazabilidad técnica y los motores de descubrimiento.
- **Capa de Datos y Optimización:** El modelo lógico adopta una estrategia de persistencia heterogénea. La consistencia transaccional y la integridad referencial se delegan a una Base de Datos SQL central (accesible mediante controladores JDBC/SQL). De manera complementaria, el almacenamiento de objetos binarios masivos se integra con Amazon S3 a través de su API nativa, mientras que la optimización de lectura y entrega del feed se apoya en una base de datos en memoria (Redis) que opera como capa de caché.

Hipstagram: Vista Lógica - Arquitectura de Componentes de Software (UML)



Capa 1 — Clientes Externos (Frontend)

El sistema atiende dos tipos de clientes completamente independientes: la App Móvil de Hipstagram utilizada por el rol USER para todas las funciones sociales, y el Panel de Control Web exclusivo para el rol ADMIN. Ambos se comunican con el backend únicamente mediante REST API sobre HTTPS/JSON, y el Panel Admin implementa además la especificación OpenAPI/Swagger para documentación de endpoints.

Capa 2 — Puerta de Entrada y Seguridad

El API Gateway es el único componente expuesto al exterior. Centraliza dos responsabilidades críticas: la seguridad JWT (verificación de tokens, extracción de rol y aplicación de RBAC) y el enrutamiento de cada solicitud hacia el microservicio interno correspondiente. Ningún servicio de las capas inferiores es accesible directamente desde los clientes.

Capa 3 — Servicios Core

Conformada por los cinco microservicios de negocio principal, cada uno con dominio propio y bien delimitado:

- **auth-service** — gestiona registro, login, logout y ciclo de vida de tokens JWT.
- **user-service** — administra perfiles de usuario y su estado (activo/inactivo).
- **post-service** — maneja metadatos, descripciones y ciclo de vida de publicaciones.
- **media-service** — procesa y sube imágenes a Amazon S3 vía S3 API.
- **moderation-service** — ejecuta el filtrado automático de contenido antes de que una publicación sea confirmada.

Capa 4 — Servicios de Soporte y Datos

Concentra los servicios especializados y las tres capas de persistencia:

- **audit-service** — registra técnicamente todos los eventos relevantes del sistema mediante JDBC/SQL hacia PostgreSQL.
- **vote-service** — gestiona likes y dislikes con idempotencia garantizada por restricción UNIQUE en BD.
- **comment-service** — administra comentarios paginados con sanitización de contenido.
- **search-service** — implementa el feed principal ordenado por likes y la búsqueda full-text, accediendo a PostgreSQL en modo solo lectura.
- **Almacenamiento de Objetos (Amazon S3)** — repositorio de imágenes accedido vía S3 API tanto por media-service como por search-service.
- **Redis** — capa de caché para optimización del feed, reduciendo la carga sobre PostgreSQL en consultas frecuentes y repetitivas.
- **Base de Datos SQL (PostgreSQL)** — capa de persistencia relacional central, accedida por todos los servicios mediante JDBC/SQL con transacciones ACID.

Flujo general de una solicitud

La resolución de las peticiones opera bajo un patrón de delegación unidireccional estricto (Top-Down): la solicitud externa es interceptada por el *API Gateway*, el cual valida la carga útil y la enruta al microservicio *Core* pertinente. Éste coordina la lógica de negocio con los servicios de soporte requeridos y ejecuta las mutaciones o consultas sobre la capa de persistencia (SQL, S3 o Redis). Finalmente, la respuesta es serializada y propagada de forma inversa hasta culminar su ciclo de vida en el cliente de origen.

5. Diagrama de Capas

La topología arquitectónica del sistema Hipstagram se fundamenta en un patrón de diseño concéntrico estructurado en capas (*Onion Architecture* / Arquitectura Limpia). Este modelo impone una estricta regla de dependencia unidireccional: el acoplamiento fluye invariablemente desde las capas tecnológicas externas hacia el núcleo del dominio. Este aislamiento estratégico garantiza que la lógica de negocio central permanezca completamente agnóstica frente a los detalles de implementación, *frameworks* externos, interfaces de usuario y mecanismos de persistencia.

Hipstagram: Diagrama de Arquitectura de Capas - Estilo Cebolla / Stacked



Capa 1 — Presentación / Interfaz de Usuario (azul)

Constituye el perímetro de interacción externa del sistema. Alberga el API Gateway como único punto de contacto, los Controladores REST encargados de la resolución de endpoints (autenticación, gestión de publicaciones, moderación y búsquedas), y los canales de comunicación asíncrona mediante WebSockets/SignalR. Adicionalmente, engloba los clientes de consumo frontend: la aplicación móvil desarrollada en React Native (rol USER) y el panel administrativo web en React (rol ADMIN). Esta capa carece absolutamente de lógica de negocio; su responsabilidad exclusiva es interceptar peticiones, delegar el procesamiento a la capa subyacente y retornar respuestas formateadas (serialization).

Capa 2 — Aplicación (verde)

Actúa como el orquestador del flujo de control del sistema. Encapsula y coordina los casos de uso específicos de la plataforma (tales como la publicación de imágenes, la emisión de votos con control de idempotencia y los flujos de moderación). Opera mediante la manipulación de Objetos de Transferencia de Datos (DTOs) para estandarizar los contratos de entrada y salida, asegurando que las entidades del dominio no se expongan directamente. Aquí reside la lógica de validación

transaccional y los servidores de aplicación (Node.js/NestJS), traduciendo las solicitudes del usuario en comandos ejecutables sobre el modelo de negocio.

Capa 3 — Infraestructura (naranja)

Centraliza el acoplamiento tecnológico y la integración con proveedores de servicios externos (IaaS/PaaS). Implementa los adaptadores concretos para la persistencia relacional en PostgreSQL, la gestión de almacenamiento de objetos binarios en la nube (Amazon S3 / Google Cloud Storage), y las bases de datos en memoria (Redis) para la optimización de cachés del feed. Asimismo, gestiona los clientes para APIs de inteligencia artificial de terceros (Azure/Google) destinadas a la moderación automática de contenido, junto con los componentes transversales de logging, telemetría y mensajería. Su diseño garantiza que la sustitución de cualquier tecnología externa no provoque mutaciones en las capas internas del software.

Capa 4 — Dominio (morada — Capa Central)

Representa el corazón inmutable de la arquitectura. Es un componente estrictamente aislado, carente de dependencias hacia bases de datos, librerías externas o protocolos de red. Define el modelo conceptual del sistema mediante Entidades con identidad propia (Usuario, Post, Imagen, Voto, Comentario) y Objetos de Valor descriptivos (JWT, *hashes* criptográficos). Concentra las reglas de negocio invariables que rigen la plataforma, gestiona Excepciones de Dominio para el control semántico de errores y propaga Eventos de Dominio para notificar cambios de estado significativos a otros componentes del ecosistema.

6. Descripción de Seguridad y Credenciales

La estrategia de seguridad del ecosistema Hipstagram se fundamenta en el paradigma de Defensa en Profundidad (Defense in Depth), estructurando múltiples capas de mitigación superpuestas para garantizar la resiliencia del sistema ante posibles vectores de ataque. Este enfoque arquitectónico asegura que la vulneración de un control perimetral no comprometa la integridad de la infraestructura subyacente. De manera complementaria, el diseño operativo adopta un modelo de Confianza Cero (Zero Trust), estableciendo que ninguna solicitud, ya sea de origen interno o externo, posee confiabilidad implícita. En consecuencia, toda interacción transversal dentro del ecosistema de microservicios exige una validación criptográfica explícita y la verificación estricta de identidad antes de conceder el acceso a los recursos protegidos.

6.1 Hashing de Contraseñas con bcrypt

Al momento del registro, la contraseña del usuario nunca se almacena en texto plano. El sistema utiliza bcrypt con un factor de costo de 12 rondas para generar el hash antes de persistirlo. A diferencia de algoritmos de propósito general como SHA-256, bcrypt incorpora un factor de costo que determina la complejidad computacional, haciéndolo resistente a ataques de diccionario y fuerza bruta incluso con hardware especializado (GPUs, ASICs).

Parámetro	Valor configurado	Implicación de seguridad
Algoritmo	bcrypt	Diseñado específicamente para contraseñas, no para velocidad. Ralentiza ataques masivos.
Factor de costo (rounds)	12	~300ms por hash en hardware moderno. Impracticable para ataques a escala.
Salt	Generado automáticamente por bcrypt	Único por contraseña. Previene rainbow table attacks.
Longitud del hash resultante	60 caracteres	Formato estándar \$2b\$12\$... almacenado en password_hash (VARCHAR 255).
Verificación	bcrypt.compare()	Comparación en tiempo constante. Previene timing attacks.

6.2 Ciclo de Vida de los Tokens JWT

El sistema implementa un esquema de doble token para la gestión de sesiones: el Access Token de corta duración (15 minutos) se incluye en el header Authorization: Bearer <token> de cada solicitud. El Refresh Token de larga duración (7 días) se almacena en una cookie HttpOnly, inaccesible para JavaScript del navegador, y se utiliza para obtener un nuevo Access Token sin requerir re-autenticación.

Evento	Access Token	Refresh Token	Acción del sistema
Login exitoso	Generado (15 min)	Generado (7 días)	Ambos enviados al cliente. refreshToken insertado en BD.
Request normal	Enviado en Authorization header	En Cookie HttpOnly	Gateway valida Access Token con jwt.verify().
Access Token expirado	Expirado	Válido (< 7 días)	Cliente llama a POST /auth/refresh. Gateway reenvía a auth-service.
Refresh exitoso	Nuevo token generado	Rotado (nuevo token)	Viejo refresh invalidado en BD. Nuevo par enviado al cliente.
Logout	Descartado en cliente	Invalidado en BD	Blacklist del refresh token en tabla refresh_tokens.
Refresh Token expirado	—	Expirado	Usuario debe hacer login nuevamente.

El payload del JWT contiene: { userId, role, iat, exp }. No contiene información sensible como contraseñas ni datos personales completos.

6.3 Control de Acceso por Roles (RBAC)

El sistema implementa Control de Acceso Basado en Roles (RBAC). Cada endpoint del API Gateway es protegido por un middleware requireRole() que extrae el rol del JWT ya verificado y lo compara con los roles permitidos para esa ruta. Existen dos roles:

Rol	Descripción	Permisos principales
USER	Usuario final registrado	Publicar, eliminar propia publicación, votar, comentar, buscar, ver feed, ver perfil propio y público.
ADMIN	Administrador del sistema	Todo lo de USER + moderar publicaciones, gestionar usuarios (activar/desactivar), administrar palabras prohibidas, consultar audit_logs.

6.4 Validación y Sanitización de Inputs

Toda entrada de datos proveniente del cliente se valida en dos capas independientes para garantizar que ningún dato malicioso llegue a la base de datos.

Campo	Capa	Validación aplicada y razón
Descripción de publicación	Gateway + Servicio	Máx. 128 caracteres, sin etiquetas HTML. Previene XSS y cumple regla de negocio.
Hashtags	Gateway + BD	Solo alfanuméricos y #, validados contra lista banned_words. Filtrado de contenido.
Correo electrónico	Gateway	Formato RFC 5322 + unicidad en BD. Integridad de datos.
Contraseña	Gateway	Mínimo 8 caracteres con combinación requerida. Estándar NIST 800-63B.
Tipo de voto	Servicio + BD	Solo valores 'LIKE' o 'DISLIKE' validados por CHECK en BD. Integridad referencial.
Comentario	Gateway + DOMPurify	Máx. 500 chars, sanitización HTML con DOMPurify. Previene XSS almacenado.

6.5 Gestión de Secretos y Variables de Entorno

Ningún secreto se almacena en el repositorio Git. Todos los valores sensibles se gestionan mediante AWS Secrets Manager en producción y variables de entorno locales en desarrollo, siguiendo el patrón .env.example en el repositorio.

Variable	Descripción	Ambiente	Gestión
JWT_SECRET	Clave para firmar Access Tokens	Todos	AWS Secrets Manager
JWT_REFRESH_SECRET	Clave para firmar Refresh Tokens	Todos	AWS Secrets Manager
DB_HOST	Host de PostgreSQL (AWS RDS endpoint)	Staging/Prod	ECS Task Definition
DB_PASSWORD	Contraseña de base de datos	Todos	Secrets Manager (nunca en Git)
S3_BUCKET	Nombre del bucket de imágenes	Staging/Prod	Variable de entorno AWS
AWS_ACCESS_KEY_ID	Clave de acceso AWS (CI/CD)	CI/CD	Jenkins Credentials Store
SONAR_TOKEN	Token de autenticación SonarQube	CI/CD	Jenkins Credentials Store

6.6 Checklist OWASP Top 10 — Estado de Implementación

#	Riesgo OWASP	Estado	Control implementado
A01	Broken Access Control	✓ Mitigado	RBAC con middleware JWT por endpoint. ADMIN endpoints bloqueados para USER. DELETE solo por autor del recurso.
A02	Cryptographic Failures	✓ Mitigado	bcrypt costo 12 para contraseñas. HTTPS/TLS 1.3 en todos los canales. Secretos en AWS Secrets Manager. Sin datos sensibles en logs.
A03	Injection	✓ Mitigado	Queries parametrizadas en PostgreSQL (\$1,\$2...). Sanitización DOMPurify. Validación Joi/Zod en Gateway.
A04	Insecure Design	✓ Mitigado	Modelo de amenazas definido. Moderación antes de publicación. Separación de roles en BD y aplicación.
A05	Security Misconfiguration	✓ Mitigado	Sin secretos en Git (.gitignore + pre-commit hook). CORS configurado. Puertos internos no expuestos. .env en .gitignore.
A06	Vulnerable Components	✓ Mitigado	npm audit en CI/CD. Escaneo con Trivy en build Docker. Dependencias actualizadas con Dependabot.
A07	Auth and Auth Failures	✓ Mitigado	JWT con expiración 15min. Refresh token HttpOnly. Auditoría de login fallido. Sin tokens en URL. Blacklist activa.
A08	Software Data Integrity	⚠ Parcial	Imágenes Docker verificadas. Sin SRI en CDN externo aún. Plan de mejora completa en Fase 3.
A09	Security Logging Failures	✓ Mitigado	Tabla audit logs con timestamp, IP, usuario, acción, entidad. Visible solo por ADMIN. Indexada por fecha.
A10	SSRF	✓ Mitigado	URLs de imagen validadas en media-service. Sin proxy genérico de URLs externas. Lista blanca de dominios S3.

7. Diagrama de Seguridad

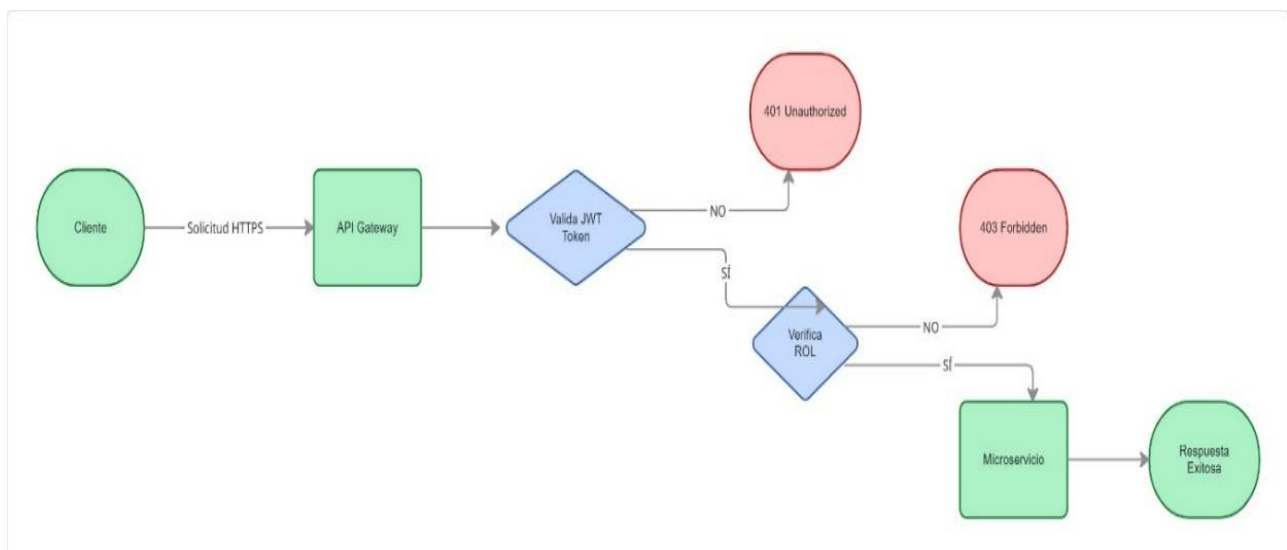
La protección perimetral e interna del ecosistema se fundamenta en el paradigma de Defensa en Profundidad (*Defense in Depth*), garantizando que las capas de red, enrutamiento, aplicación y persistencia implementen controles de mitigación autónomos. La abstracción visual describe el ciclo de vida de una petición externa, evidenciando las barreras de validación secuenciales que debe superar antes de interactuar con la lógica de negocio.

7.1 Flujo de Autenticación y Autorización

El siguiente diagrama describe el flujo completo que sigue una solicitud desde el cliente hasta el microservicio destino, pasando por los controles de seguridad implementados.

Análisis del Proceso

- **Solicitud HTTPS:** El cliente inicia la comunicación mediante un canal cifrado TLS, asegurando la confidencialidad de los datos en tránsito.
- **Intercepción en API Gateway:** Actúa como el único punto de entrada (Single Entry Point), centralizando la lógica de validación para evitar la dispersión de reglas de seguridad.
- **Validación Estricta de JWT:** Se verifica la firma criptográfica del token. Si el token ha sido manipulado o ha expirado, se retorna un error **401 Unauthorized**.
- **Autorización Basada en Roles (RBAC):** Una vez confirmada la identidad, el sistema inspecciona los *claims* del token para verificar si el usuario posee el "Rol" necesario para la operación. De lo contrario, se emite un **403 Forbidden**.

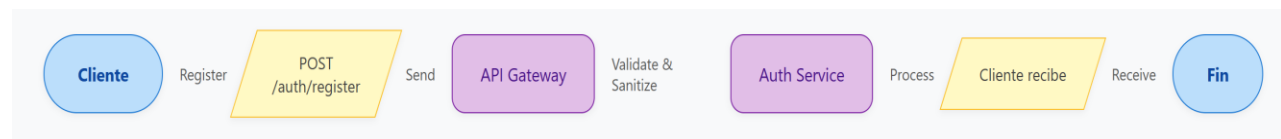


7.2 Flujo de Registro e Inicio de Sesión

El flujo transaccional para la creación de identidades se inicializa cuando el cliente emite una petición HTTP POST dirigida al endpoint /auth/register. Esta solicitud perimetral es interceptada por el API Gateway, el cual actúa como primera línea de defensa ejecutando políticas de validación estructural y sanitización de datos (previniendo vectores de ataque como la inyección de código o payloads maliciosos). Una vez inspeccionada y purgada la información, la petición es delegada al microservicio especializado (Auth Service), responsable de procesar la lógica de negocio, aplicar el hashing criptográfico y consolidar el registro. El ciclo culmina con la entrega de la respuesta transaccional de vuelta al cliente. Este procedimiento de mitigación en capas asegura que la generación de identidades y el aprovisionamiento de credenciales cumplan con los más rigurosos estándares de seguridad de la industria.

Diagrama-Proceso de registro

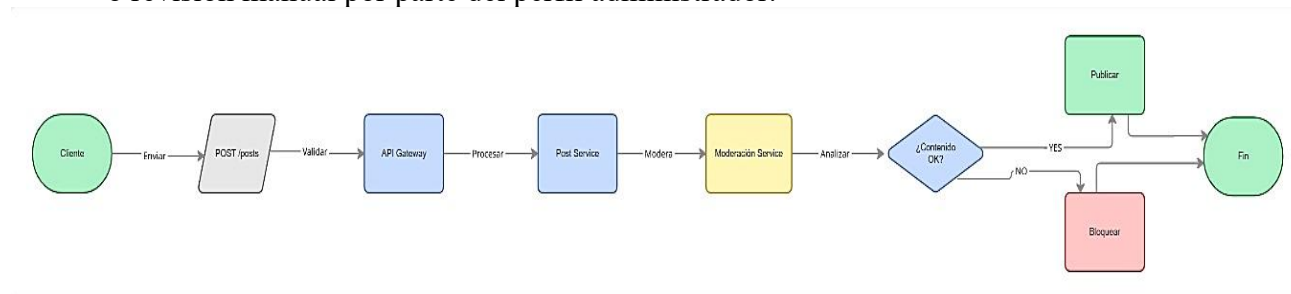
- **Endpoint:** POST /auth/register.
- **Sanitización (Validate & Sanitize):** El API Gateway limpia los datos de entrada para prevenir ataques de Cross-Site Scripting (XSS) e inyección SQL antes de que lleguen al Auth Service.
- **Persistencia Segura:** Las contraseñas se procesan mediante un algoritmo de hashing (Bcrypt) antes de almacenarse, asegurando que incluso en una fuga de datos, las claves permanezcan cifradas.



7.3 Flujo de Publicación con Moderación

En esta arquitectura, el perímetro de seguridad no se limita exclusivamente al control de acceso e identidad, sino que se extiende a la preservación de la integridad, calidad y cumplimiento normativo del contenido que ingresa a la plataforma. El diagrama ilustra el *pipeline* de validación secuencial que atraviesa toda nueva publicación:

- **Validación Estricta de Autoría:** Tras la intercepción y autorización en el *API Gateway*, la carga útil de la petición (POST /posts) es enrutada al Post Service. Este componente ejecuta una validación de coherencia transaccional, garantizando que el identificador del usuario que emite la solicitud coincida inequívocamente con el *claim* de identidad sellado criptográficamente en el JWT, mitigando así cualquier intento de suplantación (*spoofing*).
- **Orquestación del Pipeline de Moderación:** Previo a su consolidación pública, el flujo de ejecución delega el análisis de los datos al Moderation Service. Este microservicio actúa como un filtro automatizado de cumplimiento normativo, evaluando los metadatos (descripción y *hashtags*) contra los diccionarios de restricción del sistema y analizando el objeto multimedia (vía AWS Rekognition) para interceptar material inapropiado.
- **Mutación de Estado Transaccional:** La resolución arrojada por el motor de moderación determina el estado final del registro en la capa de persistencia. Si el algoritmo valida la integridad del contenido (resolución YES), el sistema autoriza la transacción y la publicación adquiere un estado activo (PUBLICADO), haciéndose visible inmediatamente en el *feed*. Por el contrario, si se detecta una infracción a las políticas (resolución NO), el sistema aplica un aislamiento preventivo; la transacción finaliza mutando el registro a un estado BLOQUEADO, restringiendo su exposición pública y dejándolo sujeto a un posible descarte o revisión manual por parte del perfil administrador.



8. Vista de Implementación

8.1 Descripción General

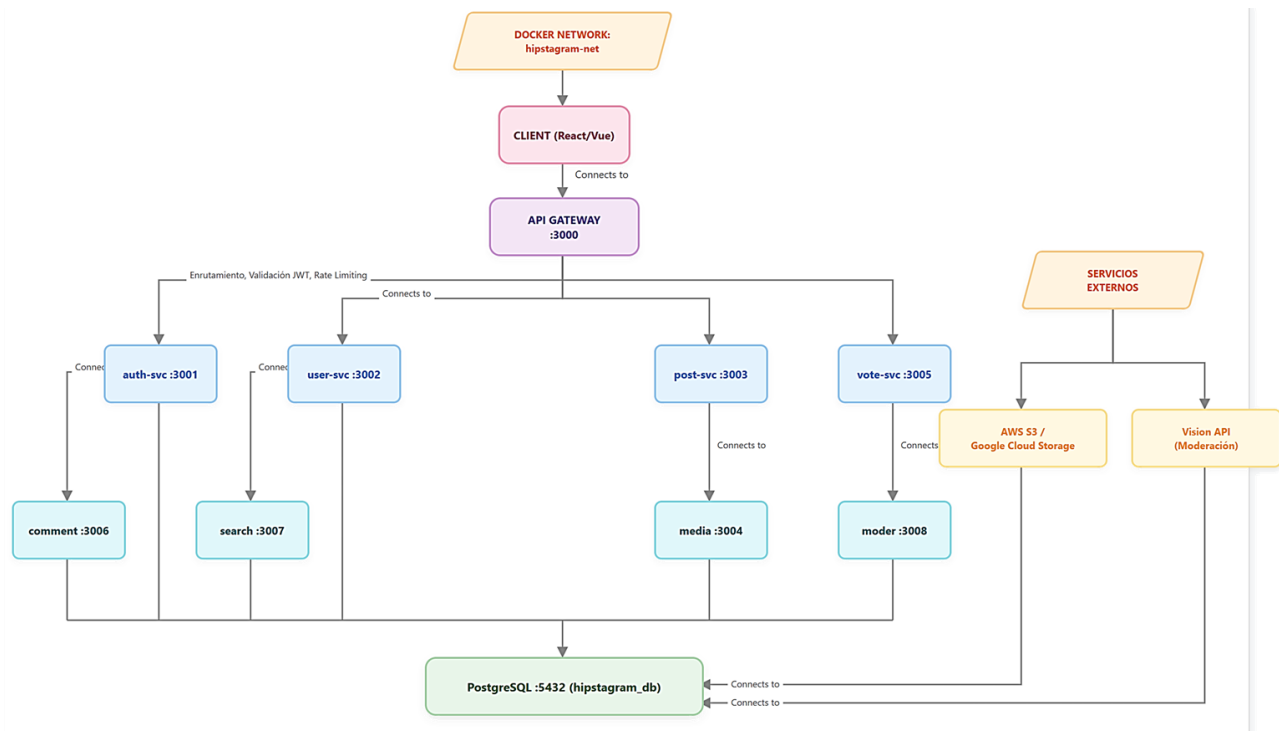
La implementación de Hipstagram se basa en una arquitectura de microservicios contenerizada. Cada componente se ejecuta como un servicio independiente dentro de un contenedor Docker, lo que permite aislar dependencias, mantener consistencia entre entornos y simplificar la administración. La infraestructura completa se define mediante Docker Compose (desarrollo) y AWS ECS Fargate (producción), siguiendo el principio de Infrastructure as Code (IaC).

El sistema sigue el patrón BFF (Backend for Frontend), donde el API Gateway actúa como único punto de entrada y centraliza autenticación, autorización, enrutamiento y rate-limiting. La comunicación entre servicios ocurre exclusivamente por HTTP/REST sobre la red interna Docker, nunca expuesta al exterior.

8.2 Catálogo de Microservicios — Especificación Completa

Microservicio	Puerto	Responsabilidad	Endpoints principales	Tablas BD
api-gateway	:3000	Entrada, JWT, RBAC, routing, rate-limit, CORS, logging	ALL /api/v1/*	Ninguna (stateless)
auth-service	:3001	Registro, login, logout, refresh token, blacklist	POST /register, /login, /logout, /refresh	users, refresh_tokens, audit_logs
user-service	:3002	Perfiles, activación/desactivación ADMIN	GET /me, GET /:id, PATCH /:id/status	users, audit_logs
post-service	:3003	CRUD publicaciones, feed, Explorar, hashtags	POST /, DELETE /:id, GET /feed, GET /explore	posts, hashtags, post_hashtags, audit_logs
media-service	:3004	Subida a S3, Rekognition, URLs firmadas	POST /upload, DELETE /:key	Ninguna (stateless)
vote-service	:3005	Like/dislike con upsert atómico, control duplicidad	POST /votes/:postId	votes, posts, audit_logs
comment-service	:3006	Comentarios paginados, sanitización, FK RESTRICT	POST /comments/:postId, GET /comments/:postId	comments, audit_logs
search-service	:3007	Búsqueda full-text por hashtag y texto libre	GET /search?q=, GET /search/hashtag?q=	posts, hashtags (solo lectura)
moderation-service	:3008	Filtrado hashtags prohibidos (interno)	POST /moderate (solo red interna)	banned_words, audit_logs

8.3 Diagrama de Implementación



Docker compose-configuracion Principal

El archivo docker-compose.yml constituye el manifiesto de Infraestructura como Código (IaC) del proyecto. Sus componentes clave garantizan la resiliencia y la escalabilidad local:

- **Gestión de Dependencias:** El uso de la directiva `depends_on` asegura que el api-gateway no inicie hasta que los servicios base (auth-service, post-service) estén en ejecución.
- **Persistencia de Estado:** Se define un volumen nombrado `pgdata` mapeado a `/var/lib/postgresql/data`, lo que garantiza que la base de datos sea persistente incluso si el contenedor es destruido.
- **Salud del Sistema (Healthchecks):** El gateway incluye un mecanismo de monitoreo automático que verifica la disponibilidad del endpoint `/health` cada 30 segundos.
- **Seguridad y Entorno:** El sistema utiliza variables de entorno externas (`{JWT_SECRET}`, `{DB_USER}`) para evitar la exposición de credenciales sensibles en el código fuente.

8.4 Estructura del Repositorio — GitFlow

```
hipstagram/ (monorepo)
├── api-gateway/ # Middleware JWT, RBAC, proxy, rate-limit
│   ├── Dockerfile
│   ├── package.json
│   └── src/
│       ├── middleware/ # verifyJWT.js, requireRole.js, rateLimiter.js
│       └── routes/ # Definición de rutas → proxy al microservicio
├── services/
│   ├── auth/ # Cada servicio: Dockerfile + package.json + src/
│   ├── user/
│   ├── post/
│   ├── media/
│   ├── vote/
│   ├── comment/
│   ├── search/
│   └── moderation/
├── frontend/ # React + Vite
│   ├── Dockerfile
│   └── src/ # components/, pages/, hooks/, api/
├── database/
│   ├── migrations/ # 001_init.sql, 002_indexes.sql ...
│   ├── roles.sql # GRANT/REVOKE — evidencia DBA
│   └── seeds/ # 001_admin.sql — datos iniciales
├── jenkins/
│   └── Jenkinsfile # Pipeline CI/CD completo
├── docker-compose.yml # Desarrollo local
├── docker-compose.prod.yml # Producción AWS
├── .env.example # Plantilla de variables (nunca .env en Git)
└── .gitignore # .env, node_modules, coverage, *.log
```

8.5 Dockerfile — Patrón Multi-Stage Build

Cada microservicio utiliza un Dockerfile con construcción en múltiples etapas (multi-stage build) para minimizar el tamaño de la imagen final y excluir herramientas de desarrollo del ambiente de producción.

Etapa 1: Builder — instala TODO incluyendo devDependencies

```
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci # Instalación limpia y reproducible
COPY . .
RUN npm run build # Transpila TypeScript si aplica
```

Etapa 2: Production — solo dependencias de producción

```
FROM node:20-alpine AS production
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production # Sin devDependencies ni herramientas de test
```

```
COPY --from=builder /app/dist ./dist
USER node          # Sin root — principio de mínimo privilegio
EXPOSE 3001        # Puerto declarativo (no mapeo real)
CMD ['node', 'dist/server.js']
```


9. Vista de Puesta en Marcha

La puesta en marcha del sistema comprende el despliegue desde el entorno de desarrollo local hasta el ambiente productivo en AWS. El proceso sigue el flujo GitFlow con CI/CD automatizado mediante Jenkins, garantizando que ningún cambio llegue a producción sin pasar por el Quality Gate de SonarQube.

9.1 Estrategia de Ambientes

Ambiente	Propósito	Quién lo usa	Infraestructura	URL / Acceso
Development	Desarrollo y pruebas unitarias	Priscila (dev)	Docker Compose local	http://localhost:3000 — Datos de prueba (seeds)
Staging / QA	Integración, pruebas de carga y revisión del cátedrático	Todo el equipo, cátedrático	AWS EC2 + RDS	https://staging.hipstagram.io — Copia anonimizada
Production	Ambiente final evaluado	Cátedrático evaluador	AWS ECS Fargate + RDS Multi-AZ	https://hipstagram.io — Datos reales del sistema

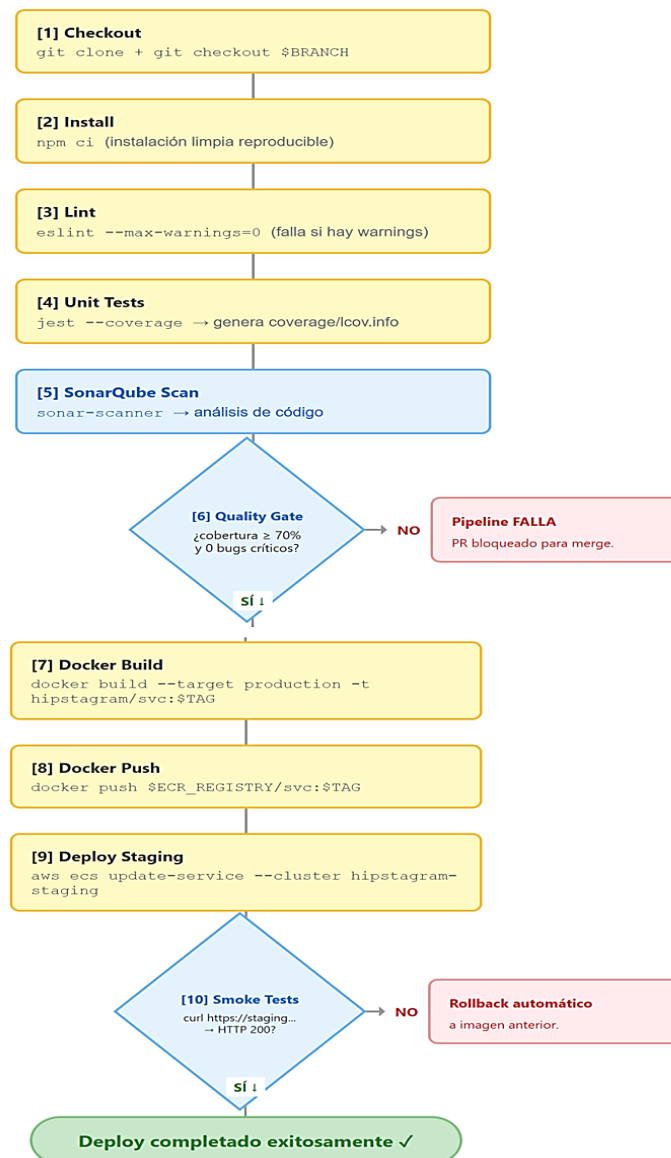
9.2 Infraestructura AWS — Detalle de Servicios

Servicio AWS	Tipo	Configuración	Costo est./mes
Application Load Balancer	Red	1 ALB, 2 AZs, SSL terminado en el ALB, redirige a ECS	~\$18 USD
ECS Fargate — API Gateway	Cómputo	0.5 vCPU, 1 GB RAM, 1 tarea (Auto Scaling Fase 3)	~\$15 USD
ECS Fargate — Microservicios (8)	Cómputo	0.25 vCPU, 512 MB RAM c/u. Total: 8 tareas independientes	~\$60 USD
RDS PostgreSQL 15	Base de datos	db.t3.micro, 20 GB SSD, Multi-AZ opcional en Fase 3	~\$15 USD
Amazon S3	Almacenamiento	Bucket hipstagram-media, acceso privado, versionado activo	~\$2 USD
Amazon ECR	Registry	9 repositorios de imágenes Docker (1 por microservicio)	~\$1 USD
CloudWatch Logs	Observabilidad	Grupos de logs por servicio, retención 30 días	~\$5 USD
AWS Secrets Manager	Seguridad	6 secretos: JWT_SECRET, JWT_REFRESH_SECRET, DB_PASSWORD, y claves IAM	~\$1 USD
TOTAL ESTIMADO	—	Ambiente de evaluación académica — instancias mínimas	~\$117 USD/mes

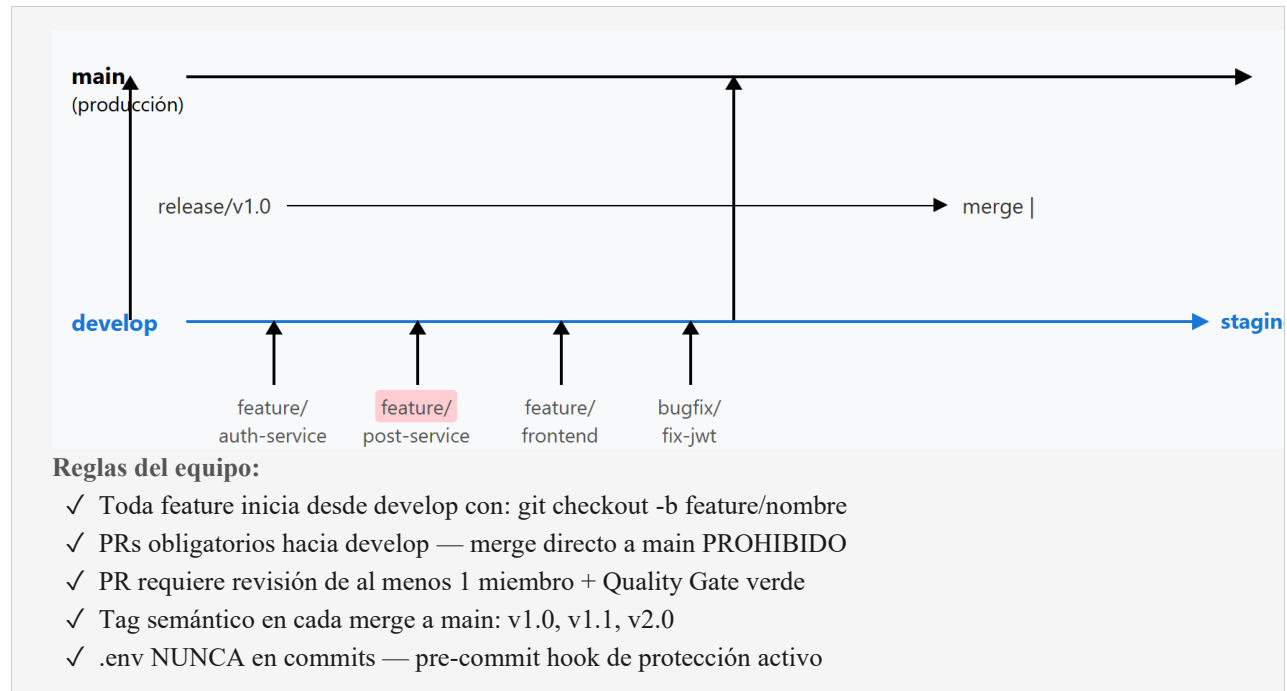
9.3 Pipeline CI/CD — Jenkinsfile

La automatización del ciclo de integración y despliegue continuo se implementa mediante Jenkins. Cada Pull Request a la rama develop activa el pipeline completo. Solo los PRs que pasan el Quality Gate de SonarQube (cobertura > 70%, 0 bugs críticos) son habilitados para merge.

Pipeline CI/CD — Etapas (Jenkinsfile)



9.4 GitFlow — Estrategia de Ramas



9.5 Pasos de Puesta en Marcha — Primera Vez

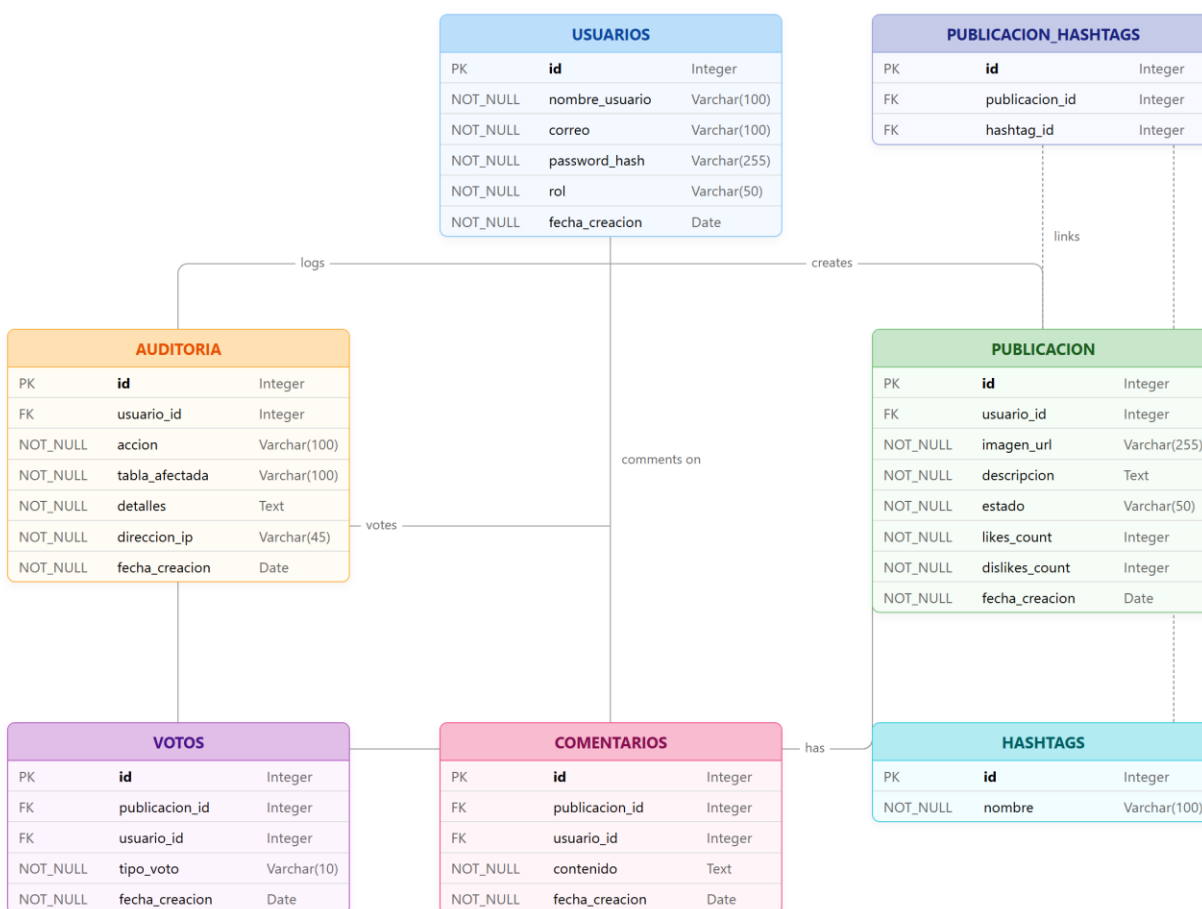
#	Acción	Comando / Detalle	Responsable
1	Clonar repositorio	<code>git clone <repo-url> && git flow init</code>	Todo el equipo
2	Copiar variables de entorno	<code>cp .env.example .env && editar con valores reales</code>	Priscila (DBA/Dev)
3	Crear BD y ejecutar migraciones	<code>docker-compose up -d postgres && psql -f database/migrations/001_init.sql</code>	Priscila (DBA/Dev)
4	Crear roles y permisos BD	<code>psql -f database/roles.sql # GRANT/REVOKE para app_role, admin_read_role, dba_role</code>	Priscila (DBA/Dev)
5	Ejecutar seeds de datos iniciales	<code>psql -f database/seeds/001_admin.sql # Crea usuario ADMIN inicial</code>	Priscila (DBA/Dev)
6	Levantar todos los servicios	<code>docker-compose up -d --build</code>	Priscila (DBA/Dev)
7	Verificar health checks	<code>curl http://localhost:3000/health → espera HTTP 200</code>	Priscila (DBA/Dev)
8	Ejecutar suite de pruebas	<code>npm test → cobertura debe ser > 70%</code>	Priscila (DBA/Dev)
9	Configurar Jenkins	Jenkins → New Item → Pipeline → SCM → apuntar a Jenkinsfile	Priscila (DevOps)
10	Configurar SonarQube	SonarQube UI → Create Project → hipstagram → Quality Gate: cobertura 70%	Priscila (DevOps)
11	Primer deploy a staging	Push a rama develop → Jenkins CD automático → <code>staging.hipstagram.io</code>	Jenkins (automático)

10. Vista de Datos

El modelo de datos de Hipstagram se compone de 9 tablas que representan los objetos del dominio del sistema. Las relaciones garantizan integridad referencial mediante claves foráneas con políticas de eliminación controladas (RESTRICT o CASCADE según corresponda). El diseño sigue las formas normales 1FN, 2FN y 3FN.

10.1 Diagrama Entidad-Relación (ERD)

El siguiente diagrama muestra las 9 entidades del sistema y sus relaciones de cardinalidad. Las líneas dobles indican claves primarias y las líneas con notación (1,N) indican relaciones de uno a muchos.



10.2 Índices y Estrategia de Optimización

Índice	Tabla y columnas	Justificación
idx_posts_likes	posts(likes_count DESC)	Acelera el feed ordenado por popularidad y el modo Explorar. Consulta más frecuente del sistema.
idx_posts_status	posts(estado)	Filtra publicaciones PUBLICADAS vs PENDIENTES/BLOQUEADAS en todas las consultas del feed.
idx_post_hashtags_htag	post_hashtags(hashtag_id)	Acelera búsqueda por hashtag que hace JOIN sobre esta tabla.
idx_audit_logs_fecha	audit_logs(fecha_creacion DESC)	El panel ADMIN filtra logs por rango de fechas. Sin índice, sería un full scan de tabla grande.
idx_comments_post	comments(publicacion_id)	Lista comentarios por publicación. FK ya crea un índice implícito en muchos motores, pero se declara explícitamente.
idx_posts_tsvector	posts USING GIN (to_tsvector(description))	Índice GIN para búsqueda full-text. Sin este índice, to_tsvector requiere seq scan de toda la tabla posts.

11. Diccionario de Datos

El Diccionario de Datos constituye el artefacto de referencia oficial y la única fuente de verdad (Single Source of Truth) para el modelo físico de persistencia relacional del ecosistema Hipstagram. Este catálogo documenta con estricto rigor técnico la topología del esquema de base de datos, detallando la semántica operativa de cada atributo, los tipos de datos nativos implementados (PostgreSQL), y las reglas de integridad estructural (Llaves Primarias, Llaves Únicas y restricciones de dominio mediante Constraints).

Para garantizar la coherencia evolutiva del software y facilitar los flujos de Integración Continua (CI), la arquitectura tabular aquí especificada se mantiene estrictamente sincronizada con los scripts de Lenguaje de Definición de Datos (DDL), los cuales se encuentran versionados bajo control de código fuente en el directorio database/migrations/ del repositorio principal.

11.1 Tabla: users

#	Columna	Tipo SQL	Nulo	Descripción / Restricciones
1	id	SERIAL	NO	PK. Identificador autoincremental. Nunca se reutiliza.
2	nombre_usuario	VARCHAR(50)	NO	UK. Handle visible (@usuario). Solo alfanumérico, guiones y guiones bajos.
3	correo	VARCHAR(100)	NO	UK. Email de autenticación. Formato RFC 5322. Único en el sistema.
4	password_hash	VARCHAR(255)	NO	Hash bcrypt factor 12. Formato \$2b\$12\$... NUNCA texto plano.
5	rol	VARCHAR(10)	NO	DEFAULT 'USER'. Valores: 'USER' o 'ADMIN'. CHECK constraint.
6	activo	BOOLEAN	NO	DEFAULT TRUE. FALSE = usuario no puede iniciar sesión. ADMIN puede desactivar.
7	fecha_creacion	TIMESTAMP	NO	DEFAULT NOW(). UTC. Auditoría de creación de cuenta.

11.2 Tabla: posts

#	Columna	Tipo SQL	Nulo	Descripción / Restricciones
1	id	SERIAL	NO	PK autoincremental.
2	usuario_id	INT	NO	FK → users.id ON DELETE RESTRICT. No se puede eliminar usuario con publicaciones.
3	imagen_url	VARCHAR(500)	NO	URL S3. Ejemplo: https://bucket.s3.amazonaws.com/uuid.jpg
4	descripcion	VARCHAR(128)	SÍ	NULL si el usuario no escribe texto. Máx. 128 chars (regla de negocio).
5	estado	VARCHAR(20)	NO	DEFAULT 'PENDIENTE'. CHECK IN ('PENDIENTE','PUBLICADO','BLOQUEADO').
6	likes_count	INT	NO	DEFAULT 0. CHECK >= 0. Actualizado atómicamente en transacción de voto.
7	dislikes_count	INT	NO	DEFAULT 0. CHECK >= 0. Actualizado atómicamente en transacción de voto.
8	fecha_creacion	TIMESTAMP	NO	DEFAULT NOW(). UTC. Desempate en feed si mismo número de likes.

11.3 Tabla: votes

#	Columna	Tipo SQL	Nulo	Descripción / Restricciones
1	id	SERIAL	NO	PK autoincremental.
2	publicacion_id	INT	NO	FK → posts.id ON DELETE CASCADE. Al eliminar post, sus votos se eliminan.
3	usuario_id	INT	NO	FK → users.id ON DELETE CASCADE. Al eliminar usuario, sus votos se eliminan.
4	tipo_voto	VARCHAR(10)	NO	CHECK IN ('LIKE','DISLIKE'). Solo estos dos valores aceptados.
5	fecha_creacion	TIMESTAMP	NO	DEFAULT NOW(). Marca de tiempo del voto o del último cambio de reacción.
—	UNIQUE constraint	—	—	UNIQUE(usuario_id, publicacion_id). Un usuario solo puede tener UN voto activo por publicación.

11.4 Tabla: comments

#	Columna	Tipo SQL	Nulo	Descripción / Restricciones
1	id	SERIAL	NO	PK autoincremental.
2	publicacion_id	INT	NO	FK → posts.id ON DELETE RESTRICT. No se puede eliminar post con comentarios. Backend captura SQLSTATE 23503 → HTTP 409.
3	usuario_id	INT	NO	FK → users.id ON DELETE RESTRICT. Protege integridad del historial.

4	contenido	TEXT	NO	Texto del comentario. Sanitizado con DOMPurify. Frontend limita a 500 chars.
5	fecha_creacion	TIMESTAMP	NO	DEFAULT NOW(). Ordenado ASC en listado de comentarios.

11.5 Tablas: hashtags y post_hashtags

#	Columna	Tipo SQL	Nulo	Descripción / Restricciones
—	hashtags	—	—	Catálogo de hashtags únicos del sistema.
1	id	SERIAL	NO	PK autoincremental.
2	nombre	VARCHAR(50)	NO	UNIQUE. Nombre normalizado incluyendo #. Upsert: INSERT ... ON CONFLICT DO NOTHING.
—	post_hashtags	—	—	Relación N:M entre posts y hashtags.
1	publicacion_id	INT	NO	PK (parte 1). FK → posts.id ON DELETE CASCADE.
2	hashtag_id	INT	NO	PK (parte 2). FK → hashtags.id ON DELETE CASCADE.

11.6 Tabla: audit_logs

#	Columna	Tipo SQL	Nulo	Descripción / Restricciones
1	id	SERIAL	NO	PK autoincremental. Nunca reutilizado.
2	usuario_id	INT	SÍ	FK → users.id ON DELETE SET NULL. NULL si el usuario fue eliminado (preserva historial).
3	accion	VARCHAR(100)	NO	Código del evento: USER_REGISTERED, LOGIN_SUCCESS, LOGIN_FAILED, POST_CREATED, POST_DELETED, VOTE_LIKE, VOTE_DISLIKE, COMMENT_CREATED, POST_APPROVED, POST_BLOCKED, BANNED_WORDS_UPDATE.
4	tabla_afectada	VARCHAR(50)	SÍ	Nombre de la tabla principal afectada. Permite filtrar por entidad.
5	detalles	TEXT (JSON)	SÍ	JSON con datos adicionales. Ejemplo: {"post_id":42,"hashtags":["#viaje"]}. Sin datos sensibles.
6	direccion_ip	VARCHAR(45)	SÍ	IP de origen. Soporte IPv4 (15 chars) e IPv6 (39 chars). Header X-Forwarded-For.
7	fecha_creacion	TIMESTAMP	NO	DEFAULT NOW(). UTC. Indexada para consultas por rango en panel ADMIN.

11.7 Tablas: refresh_tokens y banned_words

#	Columna	Tipo SQL	Nulo	Descripción / Restricciones
—	refresh_tokens	—	—	Tokens de sesión larga duración. Permite invalidación (logout, rotación).
1	id	SERIAL	NO	PK autoincremental.
2	user_id	INT	NO	FK → users.id ON DELETE CASCADE.
3	token	TEXT	NO	UNIQUE. JWT firmado del refresh token.
4	expires_at	TIMESTAMP	NO	Expiración absoluta. auth-service verifica que no haya expirado antes de renovar.
—	banned_words	—	—	Lista de palabras/hashtags prohibidos. Gestionada por ADMIN.
1	id	SERIAL	NO	PK autoincremental.
2	word	VARCHAR(100)	NO	UNIQUE. Palabra o hashtag prohibido. El moderation-service consulta esta tabla en cada publicación.
3	created_by	INT	SÍ	FK → users.id (ADMIN que la agregó). Trazabilidad de gestión de palabras prohibidas.

12. Definición de Roles de la Aplicación

El ecosistema implementa una estrategia de autorización bifurcada, gestionando los privilegios en dos estratos arquitectónicos estrictamente independientes. En la capa de aplicación, el Control de Acceso Basado en Roles (RBAC) se orquesta mediante los claims de identidad sellados en los JSON Web Tokens (JWT) y el estado lógico validado contra la tabla users. Paralelamente, en la capa de infraestructura, la persistencia relacional se blinda configurando roles de base de datos nativos a través de sentencias DCL (GRANT/REVOKE en PostgreSQL). La convergencia de ambos mecanismos de control opera de manera complementaria, materializando de forma estricta el principio de mínimo privilegio (Principle of Least Privilege) y garantizando una mitigación robusta frente a posibles vulnerabilidades de escalamiento de privilegios.

12.1 Roles de Aplicación

Rol	Tipo de usuario	Permisos y capacidades
USER	Usuario registrado final	Registro y login. Publicar/eliminar propia publicación (sujeto a moderación). Dar like/dislike a publicaciones ajenas. Comentar. Buscar por hashtag y texto libre. Ver feed y modo Explorar. Ver perfil propio y público. Logout.
ADMIN	Administrador del sistema	Todo lo de USER. Aprobar/rechazar/bloquear publicaciones. Activar/desactivar cuentas de usuario. Gestionar lista de palabras prohibidas (JSON). Consultar audit_logs completo con filtros. Ver métricas del sistema.

Los roles se incluyen en el payload del JWT al momento del login. El API Gateway lee el campo role del JWT ya verificado para aplicar RBAC sin consultar la base de datos en cada petición, garantizando rendimiento y seguridad.

12.2 Endpoints por Rol — USER

Módulo	Acción	Método	Endpoint	Body/Params	Respuesta
Auth	Registro	POST	/auth/register	{nombre,correo,password}	201 {user}
Auth	Login	POST	/auth/login	{correo,password}	200 {accessToken}+Cookie
Auth	Logout	POST	/auth/logout	Cookie: refreshToken	200 {message}
Auth	Refresh token	POST	/auth/refresh	Cookie: refreshToken	200 {accessToken}
Posts	Crear publicación	POST	/posts	FormData {imagen,desc,hashtags}	201 {post}
Posts	Eliminar propia	DELETE	/posts/:id	—	200 / 403 si no autor
Posts	Ver feed	GET	/posts/feed?page=1&limit=10	—	200 {posts[],total,page}
Posts	Modo Explorar	GET	/posts/explore?page=1	—	200 {posts[] por likes}

Votos	Like/Dislike	POST	/votes/:postId	{tipo_voto:'LIKE' 'DISLIKE'}	200 {likes,dislikes}
Comentarios	Crear comentario	POST	/comments/:postId	{contenido}	201 {comentario}
Comentarios	Listar	GET	/comments/:postId?page=1	—	200 {comentarios[],total}
Búsqueda	Por hashtag	GET	/search/hashtag?q=#guate	—	200 {posts[]}
Búsqueda	Texto libre	GET	/search?q=xela&page=1	—	200 {posts[]}
Perfil	Ver mi perfil	GET	/users/me	—	200 {user,stats}

12.3 Endpoints Exclusivos — ADMIN

Módulo	Acción	Método	Endpoint	Descripción
Moderación	Posts PENDIENTES	GET	/admin/posts?estado=PENDIENTE	Listado paginado para revisión manual.
Moderación	Aprobar	PATCH	/admin/posts/:id/approve	Cambia estado a PUBLICADO. Registra en audit_logs.
Moderación	Rechazar	PATCH	/admin/posts/:id/reject	Cambia estado a BLOQUEADO. Registra en audit_logs.
Usuarios	Listar	GET	/admin/users?q=email&page=1	Búsqueda y listado paginado con filtros.
Usuarios	Activar/Desactivar	PATCH	/admin/users/:id/status	{activo:true false}. Impide login sin eliminar cuenta.
Palabras	Ver lista	GET	/admin/banned-words	Retorna JSON actual de palabras/hashtags prohibidos.
Palabras	Actualizar	PUT	/admin/banned-words	Reemplaza lista completa. Registro en audit_logs.
Auditoría	Ver log	GET	/admin/audit?fecha=&usuario=&accion=	Log completo filtrable. Solo visible para ADMIN.
Métricas	Dashboard	GET	/admin/metrics	Posts/día, likes/día, usuarios bloqueados, activos.

12.4 Roles de Base de Datos (DBA)

El principio de mínimo privilegio se aplica también en la capa de base de datos. Los microservicios de aplicación usan un rol (app_role) sin acceso a la tabla audit_logs. El módulo de auditoría del ADMIN usa un rol de solo lectura (admin_read_role). El rol DBA es exclusivo de scripts de migración y mantenimiento.

Rol BD	Usuario BD	Permisos	Usado por
app_role	app_user	SELECT, INSERT, UPDATE, DELETE en todas las tablas EXCEPTO audit_logs	Todos los microservicios de aplicación (auth, post, vote, comment, search, media, moderation)
admin_read_role	admin_user	SELECT en audit_logs + todas las tablas (solo lectura)	Endpoint /admin/audit en user-service. Solo cuando role='ADMIN' en JWT.
dba_role	dba_user	ALL PRIVILEGES en schema public + sequences	Scripts de migración (database/migrations/), mantenimiento y backups. NUNCA usado por la aplicación.

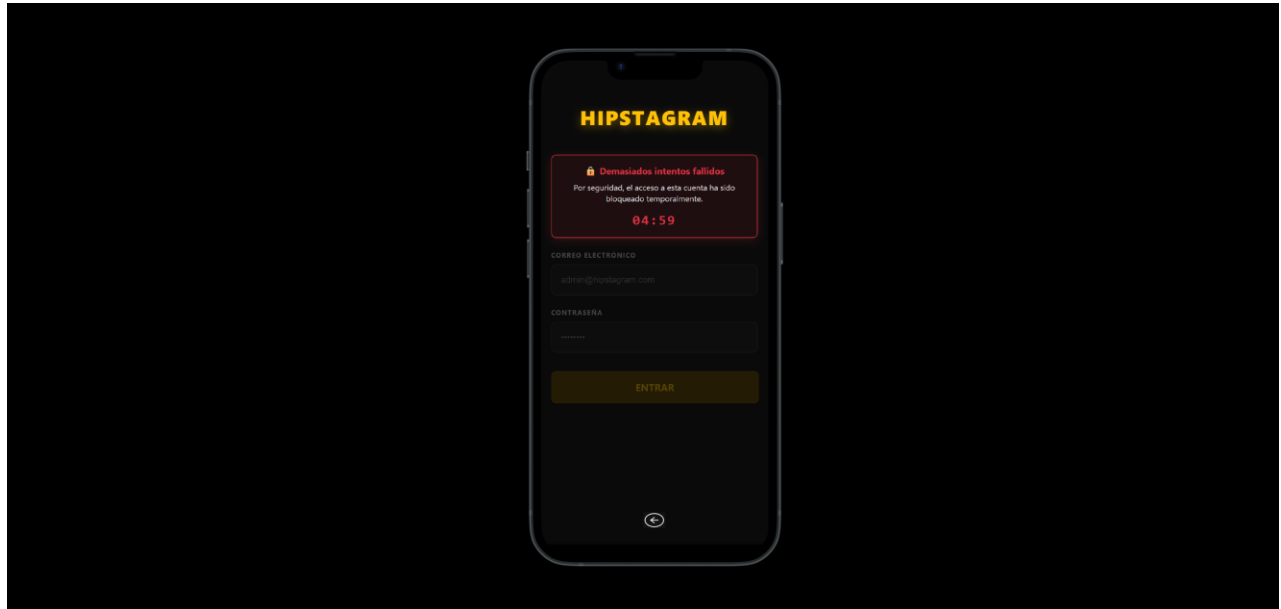
12.5 Códigos de Respuesta HTTP — Estándar del Sistema

Código	Nombre	Cuándo se usa en Hipstagram
200	OK	Solicitud procesada correctamente. GET, PATCH, DELETE exitosos.
201	Created	Recurso creado exitosamente. POST de registro, publicación o comentario.
400	Bad Request	Schema inválido, campo faltante, descripción > 128 chars, tipo_voto inválido.
401	Unauthorized	Token JWT ausente, expirado o con firma inválida.
403	Forbidden	Usuario autenticado pero sin el rol requerido. USER intentando endpoint de ADMIN.
404	Not Found	Publicación, usuario o comentario con el id especificado no existe.
409	Conflict	Correo ya registrado, FK RESTRICT violada al eliminar (SQLSTATE 23503), voto duplicado.
422	Unprocessable Entity	Datos válidos sintácticamente pero rechazados: imagen inapropiada, hashtag prohibido.
429	Too Many Requests	Más de 100 solicitudes por minuto desde la misma IP (rate-limit en API Gateway).
500	Internal Server Error	Error inesperado del servidor. Nunca se exponen detalles técnicos en producción.

13. Prototipo No Funcional

Seguridad — Bloqueo Temporal por Intentos Fallidos

El sistema implementa un mecanismo de protección contra ataques de fuerza bruta en el proceso de autenticación. Como se observa en la pantalla, tras 5 intentos fallidos consecutivos de inicio de sesión, el acceso a la cuenta queda bloqueado temporalmente durante 5 minutos, mostrando un contador regresivo visible al usuario (04:59) junto con el mensaje: *"Por seguridad, el acceso a esta cuenta ha sido bloqueado temporalmente."*



14. Alcance y Limitaciones de la Aplicación

14.1 Alcance Funcional Completo

ID	Funcionalidad	Rol	Prioridad	Fase de entrega
F-01	Registro con validación email y contraseña	USER/ADMIN	Alta	Fase 1 (documentación)
F-02	Login con JWT + Refresh Token (HttpOnly Cookie)	USER/ADMIN	Alta	Fase 1 (documentación)
F-03	Logout e invalidación de Refresh Token	USER/ADMIN	Alta	Fase 1 (documentación)
F-04	Publicar fotografía con descripción (≤128 chars) y hashtags	USER	Alta	Fase 2 (implementación)
F-05	Eliminar propia publicación (si no tiene comentarios activos)	USER	Alta	Fase 2
F-06	Feed ordenado por likes con paginación (10/página)	USER/ADMIN	Alta	Fase 2
F-07	Modo Explorar — Top Likes global, paginado	USER/ADMIN	Media	Fase 2
F-08	Like/Dislike con control de duplicidad e idempotencia	USER	Alta	Fase 2
F-09	Comentarios con paginación y sanitización XSS	USER	Alta	Fase 2
F-10	Búsqueda por hashtag exacto	USER/ADMIN	Alta	Fase 2
F-11	Búsqueda full-text libre	USER/ADMIN	Media	Fase 2
F-12	Moderación automática de imagen (AWS Rekognition)	Sistema	Alta	Fase 2
F-13	Filtrado automático de hashtags prohibidos	Sistema	Alta	Fase 2
F-14	Panel ADMIN: gestión de publicaciones (aprobar/rechazar/bloquear)	ADMIN	Alta	Fase 2
F-15	Gestión de usuarios (activar/desactivar) por ADMIN	ADMIN	Media	Fase 2
F-16	Gestión de palabras prohibidas en JSON por ADMIN	ADMIN	Media	Fase 2
F-17	Log de auditoría filtrable solo para ADMIN	ADMIN	Alta	Fase 2
F-18	CI/CD con Jenkins, SonarQube Quality Gate	DevOps	Alta	Fase 2

F-19	Despliegue en AWS con Docker y ECS Fargate	DevOps	Alta	Fase 3
F-20	Pruebas unitarias, integración y carga con evidencia	QA	Alta	Fase 3

14.2 Limitaciones Técnicas y de Alcance

Categoría	Limitación	Impacto	Posible extensión futura
Social	Sin sistema de seguidores (follow/unfollow)	Bajo	tabla follows(follower_id, followed_id) + feed personalizado
Social	Sin mensajes directos entre usuarios	Bajo	WebSockets + tabla mensajes con cifrado
Contenido	Solo 1 imagen por publicación	Bajo	Galería múltiple con tabla publicacion_imagenes
Contenido	Sin videos ni GIFs	Medio	AWS MediaConvert para transcodificación de video
Contenido	Sin stories efímeras (24h)	Bajo	Entidad story con TTL automático en BD
UX	Sin notificaciones push en tiempo real	Medio	WebSockets (Socket.io) o Server-Sent Events
UX	Sin recuperación de contraseña por email	Medio	Flujo OTP por email con Nodemailer + token temporal
Técnico	Docker Compose sin Kubernetes ni auto-scaling	Medio	ECS con Auto Scaling Groups en Fase 3
Técnico	Sin caché distribuida (Redis)	Medio	Redis para feed cache, rate-limit y blacklist de tokens

14.3 Tabla de Riesgos del Proyecto

ID	Riesgo	Prob.	Impacto	Nivel	Plan de mitigación
R-01	Sobrecarga en feed con alto volumen de datos	Media	Alto	Alto	Índice idx_posts_likes + paginación estricta + caché Redis en Fase 3
R-02	Fallo del servicio de moderación (Rekognition)	Media	Medio	Medio	Circuit breaker: si Rekognition falla, publicación queda PENDIENTE para revisión manual.
R-03	Brecha de seguridad en JWT (token robado)	Baja	Muy Alto	Alto	Tokens de 15min + refresh HttpOnly + blacklist + auditoría de IPs inusuales
R-04	Inconsistencia en contadores de votos (race condition)	Media	Medio	Medio	UPDATE atómico de contadores con subqueries de recuento directo dentro de la transacción.
R-05	Caída de base de datos PostgreSQL	Baja	Muy Alto	Alto	RDS Multi-AZ + backups automáticos diarios + reinicio automático Docker.

R-06	Pipeline CI/CD bloqueado por Quality Gate	Alta	Medio	Medio	Umbral razonable en SonarQube (70%). Código limpio con revisión de PR antes de merge.
R-07	Credenciales en el repositorio Git	Baja	Muy Alto	Alto	.gitignore estricto + pre-commit hook + revisión de PR obligatoria antes de merge a develop.
R-08	Costo de infraestructura cloud elevado	Baja	Medio	Medio	AWS Free Tier + instancias mínimas (t3.micro) + alertas de facturación + apagar fuera de evaluación.

14.4 Restricciones del Sistema

Restricción	Valor / Límite	Justificación
Descripción por publicación	Máx. 128 caracteres	Definido explícitamente en los requerimientos del proyecto.
Fotos por publicación	1 imagen	Restricción del alcance funcional Fase 1-2.
Tamaño máximo de imagen	5 MB (10 MB hard limit)	Costo de S3 y tiempo de carga en conexiones móviles.
Formatos de imagen aceptados	JPEG, PNG, WebP	Formatos con amplio soporte y buena compresión.
Comentarios por página	20 por request	Balance rendimiento / experiencia de usuario.
Publicaciones por página en feed	10 por request	Optimización de payload y tiempo de respuesta.
Hashtags por publicación	Máx. 10	Limitación recomendada para evitar spam y abuso.
Longitud mínima de contraseña	8 caracteres	Estándar mínimo NIST 800-63B.
Expiración del Access Token	15 minutos	Minimizar ventana de ataque si el token es comprometido.
Expiración del Refresh Token	7 días	Balance entre seguridad y experiencia de re-login.
Rate limit por IP	100 requests/minuto	Prevención de abuso y ataques DDoS simples.
Paginación máxima permitida	100 items/request (hard limit)	Prevenir requests excesivamente pesados al servidor.